

Rhinestone Compact Utils and Arbiters (2025-11)

Security Audit

: Rhinestone Compact Utils and Arbiters

Jan 16, 2026

Revision 1.0

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Rhinestone Compact Utils and Arbiters (2025-11) Security Audit	1
Table of Contents	2
Executive Summary	5
Audit Overview	6
Scope	6
Code Revision	7
Severity Categories	8
Status Categories	9
Finding Breakdown by Severity	10
Findings	11
Summary	12
#1 Rhinestone-2511-01 Preclaim minGas enforcement can be bypassed in ArbiterBase	14
#2 Rhinestone-2511-02 If targetOps.hashEIP712() is NO_EXEC, a malicious user can tamper with targetOps in ArbiterBase.	17
#3 Rhinestone-2511-03 By manipulating the targetOps, a malicious user can steal the funds of the filler performing the first fill on the Across Bridge	19
#4 Rhinestone-2511-04 Malicious claim function switching in Across arbiter enables first-fill fund theft	23
#5 Rhinestone-2511-05 Replayable targetOps due to missing targetChainId validation in AcrossArbiter.across_permit2_erc7579Intent_handleFill()	26
#6 Rhinestone-2511-06 Incorrect across message function used in across_permit2_multicall_handleFill()	28
#7 Rhinestone-2511-07 Incorrect notarizedChainId usage in handleCompact_ExogenousChain() for EcoArbiter and RelayArbiter	30
#8 Rhinestone-2511-08 A malicious filler can tamper with preClaimSig in PreClaimExecution._handlePreClaimOpsCompactERC7579() to deliberately cause the pre-claim call to fail	32
#9 Rhinestone-2511-09 Incorrect balance check in MultiCaller._withdrawApprovedTokens	35
#10 Rhinestone-2511-10 A malicious filler can bypass the Router and interact directly to selectively execute intents, omit targetOps, or trigger a refund.	37
#11 Rhinestone-2511-11 Collision between target address and singleCall() function signature in Caller.sol	40
#12 Rhinestone-2511-12 Reentrancy in PreClaim via routeClaim, allowing Compact claim execution in advance and causing CLAIM failure under fill-first mode	42
#13 Rhinestone-2511-13 handleNative() performs no balance validation when msg.value is 0	46
#14 Rhinestone-2511-14 Minor Suggestions	49

Executive Summary

Beginning on October 13, 2025, ChainLight conducted a security audit of Rhinestone's intent execution and bridge-arbiter smart contracts, concluding on November 7, 2025. The audit focused on validating the safety and correctness of the Claim/PreClaim lifecycle across Across/Eco/Relay paths, with particular emphasis on mandate-hash integrity, `targetOps` encoding and hashing, gas and replay protections, and resilience against manipulation and reentrancy in fill-first scenarios.

The audit revealed a total of 14 issues, categorized by severity as follows:

- Total: 14
- Critical: 3
- High: 4
- Medium: 4
- Low: 2
- Informational: 1

The commits with all patches applied are as follows:

- [https://github.com/rhinestonewtf/eco-arbiter:
9f568e459135114619e46466202abbdabeb55421](https://github.com/rhinestonewtf/eco-arbiter/commit/9f568e459135114619e46466202abbdabeb55421)
- [https://github.com/rhinestonewtf/across-arbiter:
54bfb804f35a13db68cf3e09804beac0449d2108](https://github.com/rhinestonewtf/across-arbiter/commit/54bfb804f35a13db68cf3e09804beac0449d2108)
- [https://github.com/rhinestonewtf/relay-arbiter-v2:
1c16f5ad0036598517c08254846a0deae108cd78](https://github.com/rhinestonewtf/relay-arbiter-v2/commit/1c16f5ad0036598517c08254846a0deae108cd78)
- [https://github.com/rhinestonewtf/compact-utils:
eea6a0b9761b60ba49ff6896558af1367e4acb24](https://github.com/rhinestonewtf/compact-utils/commit/eea6a0b9761b60ba49ff6896558af1367e4acb24)

Audit Overview

Scope

Name	Rhinestone Compact Utils and Arbiters (2025-11) Security Audit
Target / Version	• https://github.com/rhinestonewtf/eco-arbiter: bce7f50eeb94cfef0288b4557c5b03fcd2a72399 • https://github.com/rhinestonewtf/across-arbiter: db3b277322a25d8bc9813073bc50dd5f69445e4e • https://github.com/rhinestonewtf/relay-arbiter-v2: 1a4e40d2bdda2f5be5d0db8345d0fd4f0101af77 • https://github.com/rhinestonewtf/compact-utils: 0af9a24250de277876eeb5ecd0343036d53a7fd7
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	3	- Rhinestone-2511-02 - Rhinestone-2511-03 - Rhinestone-2511-04
High	4	- Rhinestone-2511-05 - Rhinestone-2511-08 - Rhinestone-2511-10 - Rhinestone-2511-12
Medium	4	- Rhinestone-2511-01 - Rhinestone-2511-06 - Rhinestone-2511-07 - Rhinestone-2511-11
Low	2	- Rhinestone-2511-09 - Rhinestone-2511-13
Informational	1	Rhinestone-2511-14
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	Rhinestone-2511-01	Preclaim <code>minGas</code> enforcement can be bypassed in <code>ArbitrBase</code>	Medium	Patched
2	Rhinestone-2511-02	If <code>targetOps.hashEIP712()</code> is <code>NO_EXEC</code> , a malicious user can tamper with <code>targetOps</code> in <code>ArbitrBase</code> .	Critical	Patched
3	Rhinestone-2511-03	By manipulating the <code>targetOps</code> , a malicious user can steal the funds of the filler performing the first fill on the Across Bridge	Critical	Patched
4	Rhinestone-2511-04	Malicious claim function switching in Across arbitrator enables first-fill fund theft	Critical	Patched
5	Rhinestone-2511-05	Replayable <code>targetOps</code> due to missing <code>targetChainId</code> validation in <code>AcrossArbitrator.across_permit2_erc7579Intent_handleFill()</code>	High	Patched
6	Rhinestone-2511-06	Incorrect across message function used in <code>across_permit2_multicall_handleFill()</code>	Medium	Patched
7	Rhinestone-2511-07	Incorrect <code>notarizedChainId</code> usage in <code>handleCompact</code>	Medium	Patched

#	ID	Title	Severity	Status
		<code>_ExogenousChain()</code> for <code>Ec</code> <code>oArbiter</code> and <code>RelayArbiter</code>		
8	Rhinestone-2511-08	A malicious filler can tamper with <code>preClaimSig</code> in <code>PreClaim</code> <code>Execution._handlePreClaimOpsCompactERC7579()</code> to deliberately cause the pre-claim call to fail	High	Patched
9	Rhinestone-2511-09	Incorrect balance check in <code>M</code> <code>ultiCaller._withdrawApprovedTokens</code>	Low	Patched
10	Rhinestone-2511-10	A malicious filler can bypass the Router and interact directly to selectively execute intents, omit <code>targetOps</code> , or trigger a refund.	High	Patched
11	Rhinestone-2511-11	Collision between <code>target</code> address and <code>singleCall()</code> function signature in <code>Caller.sol</code>	Medium	Patched
12	Rhinestone-2511-12	Reentrancy in <code>PreClaim</code> via <code>routeClaim</code> , allowing Compact claim execution in advance and causing CLAIM failure under fill-first mode	High	Patched
13	Rhinestone-2511-13	<code>handleNative()</code> performs no balance validation when <code>m</code> <code>sg.value</code> is 0	Low	Patched
14	Rhinestone-2511-14	Minor Suggestions	Informational	Patched

#1 Rhinestone-2511-01 Preclaim minGas enforcement can be bypassed in ArbiterBase

ID	Summary	Severity
Rhinestone-2511-01	A filler can bypass minGas enforcement in ArbiterBase because the EVM caps forwarded gas to <code>min(requested, gasleft() - floor(gasleft()/64))</code> , allowing the preclaim step to fail.	Medium

Description

In ArbiterBase, a filler can bypass minGas and execute a claim with less gas. Even if you pass minGas to a low-level call, the gas is automatically capped at `min(requested, gasleft() - floor(gasleft()/64))`. Therefore, even if the filler sets the transaction gas lower than minGas when claiming, the gas for the preclaim call is automatically changed to `gasleft() - floor(gasleft()/64)`, not minGas. Because functions like `_handlePreClaimOpsMulticall` ignore calls that fail with OOG, the claim process can still be completed using the remaining `gasleft()/64`. In this case the filler can complete a claim with less gas than minGas, which makes including minGas in the `mandateHash` meaningless because the user no longer has a guarantee that `preClaim` is invoked with minGas.

Impact

Medium

If the remaining `gasleft()/64` is still large enough to complete the claim process, a filler can deliberately provide less gas than minGas to the preclaim step and force it to fail.

Recommendation

It is recommended to revert when `gasleft() * 63 / 64 < minGas` in `ArbiterBase._compactPreClaimOps`.

References

<https://eips.ethereum.org/EIPS/eip-150>

Remediation

Patched

The issue has been resolved as recommended.

#2 Rhinestone-2511-02 If `targetOps.hashEIP712()` is `NO_EXEC`, a malicious user can tamper with `targetOps` in `ArbiterBase`.

ID	Summary	Severity
Rhinestone-2511-02	When <code>targetOps.hashEIP712()</code> returns <code>NO_EXEC</code> , an attacker can freely modify <code>targetOps</code> without changing the <code>mandateHash</code> , enabling <code>relayHash</code> manipulation in <code>AcrossArbiter</code> and arbitrary account operations in <code>SameChainArbiter</code> .	Critical

Description

In `ArbiterBase`, when the original `order.targetOps.hashEIP712()` equals `Constants.NO_EXEC`, an attacker can tamper with `targetOps`. In `SmartExecutionLib.hashEIP712AndType()`, if `_type` is not `ERC7579`, `MultiCall`, or `Calldata`, the else branch always returns `Constants.NO_EXEC`. Even if an attacker modifies a `targetOps` that originally returns `NO_EXEC`, changing its type to `Eip712Hash` and altering `targetOps[2:]` does not affect the result, because `order.targetOps.hashEIP712()` returns `Constants.NO_EXEC` regardless of the contents of `targetOps[2:]`. Consequently, the `mandateHash` remains unchanged.

A malicious filler can exploit this to manipulate `targetOps` in `SameChainArbiter.handleCompact_NotarizedChain()` and cause arbitrary actions to be executed via the user's Account Abstraction modules (`TrustedExecution.executeOpsWithoutSignature`). Furthermore, in `AcrossArbiter`, if a filler performs a first-fill on the destination chain, a malicious user can alter the message contents by tampering with `targetOps` and then execute a claim that produces a different `relayHash`, enabling theft of the first-filling filler's funds. To mitigate this issue, include `targetOps[0:2]` in the `mandateHash`, and update the logic so that when the `targetOps` type is `Eip712Hash` or `NONE`, it returns an appropriate hash of `targetOps[2:]`.

Impact

Critical

When the original return value of `targetOps.hashEIP712` is `NO_EXEC`, the following two cases can occur:

1. When a filler performs a first-fill in `AcrossArbiter`, a malicious sponsor (the user) can modify `targetOps` to change the `relayHash` and prevent the filler from receiving any settlement. In this case, the unsettled funds are refunded to the sponsor, causing a loss for the filler.
2. In `SameChainArbiter`, a malicious filler can tamper with `targetOps` and execute arbitrary actions from the sponsor (smart account) through the `TrustedExecution` module.

Recommendation

To mitigate this issue, include `targetOps[0:2]` (`Type` and `SigMode`) in the `mandateHash`, and update the logic so that when the `targetOps` type is `Eip712Hash` or `NONE`, it returns an appropriate hash of `targetOps[2:]`.

Remediation

Patched

It has been updated to compute the hash of `targetOps` via the `hashOps()` function, and `hashOps()` now performs hashing that includes the execution type and signature mode. In addition, `NONE` has been removed from `ExecutionType`, and the implementation has been patched so that, for all execution types, hashing is performed over `targetOps[2:]`.

#3 Rhinestone-2511-03 By manipulating the `targetOps` , a malicious user can steal the funds of the filler performing the first fill on the Across Bridge

ID	Summary	Severity
Rhinestone-2511-03	In Across first-fill scenarios, an attacker can modify <code>targetOps.data[2:]</code> for ERC7579-typed operations while keeping the same <code>mandateHash</code> . This changes the Across deposit message and <code>relayHash</code> , preventing the first filler from claiming their reward and causing direct financial loss.	Critical

Description

In the Across Bridge scenario where a filler performs the first fill, a malicious user on the source chain can tamper with a `targetOps` of type `ERC7579` and invoke `claim`, which (1) alters the message contents at deposit, and (2) due to the message change, alters the `relayHash` so the filler cannot claim the reward for the funds they filled.

`AcrossMessageLib._toAcrossMessageMultiCall()` performs `acrossMessage = abi.encodePacked(recipient, targetOps.data[2:])` when `targetOps` exists. Thus, any modification to `targetOps.data[2:]` changes the `acrossMessage` and therefore the `relayHash` .

However, for `ERC7579` -typed `targetOps`, an attacker can still ensure that `hashEIP712()` returns the same hash despite modifying `targetOps.data[2:]` . In `hashEIP712AndType()` , when the type is `ERC7579` , it calls `ops.onlyExecutionData().toERC7579().hashOperations()` .

Inside `toERC7579()` , the underlying `decodeBatchUnchecked()` is the root cause: given the `executionData` format, an attacker can change the dummy field in `ops.data[2:]` while still producing identical decoded pointers, causing `hashOperations()` to compute the same value. As a result, the `mandateHash` remains unchanged, even though the actual contents of `targetOps.data[2:]` were altered.

```

function decodeBatchUnchecked(bytes calldata executionData) internal pure
returns (bytes32[] calldata pointers) {
    /// @solidity memory-safe-assembly
    assembly ("memory-safe") {
        let o := add(executionData.offset, calldataload(executionData.
offset))
        pointers.offset := add(o, 0x20)
        pointers.length := calldataload(o)
    }
}

// executionData
0000000000000000000000000000000000000000000000000000000000000040 // [00]
off = 0x40
<32bytes dummy data> // [20]
dummy
00000000000000000000000000000000000000000000000000000000000001 // [40]
length = 1
00000000000000000000000000000000000000000000000000000000000020 // [60]
Execution[0] offset
000000000000000000000000000000000000000000000000000000000000111111111111111111111111111111111111111 // [80]
target
0000000000000000000000000000000000000000000000000000000000000000 // [A0]
value
0000000000000000000000000000000000000000000000000000000000000060 // [C0]
callData offset (0x60)
0000000000000000000000000000000000000000000000000000000000000000 // [E0]
callData length = 0

```

In summary, an attacker can freely modify `targetOps.data[2:]` while preserving the same `mandateHash`. This discrepancy means the Across deposit message (and therefore `relayHash`) changes, preventing the first filler from claiming their reward and causing direct financial loss.

Impact

Critical

The first filler, despite providing funds on the destination chain in a first-fill scenario, is unable to claim their reward and suffers financial loss.

Recommendation

Include the full `targetOps.data` in the hashing process so that any mutation changes the `mandateHash`.

Remediation

Patched

In `AcrossWith7579._acrossMessageFn()`, setting `acrossMessage = abi.encodePacked(recipient, claimHash);` ensures that `targetOps.data[2:]` is no longer directly embedded in `acrossMessage`. As a result, even if a different `targetOps.data[2:]` is supplied that still satisfies the same `mandateHash`, it can no longer modify `acrossMessage`.

#4 Rhinestone-2511-04 Malicious claim function switching in

Across arbiter enables first-fill fund theft

ID	Summary	Severity
Rhinestone-2511-04	A malicious user can switch to a different handler using the same allocator signature, producing a different Across deposit message and <code>relayHash</code> , which results in theft of the filler's first-fill funds.	Critical

Description

In a first-fill scenario, the filler calls `handleCompact_NotarizedChain_ERC7579()` as intended. However, a malicious user can instead invoke `handleCompact_NotarizedChain_MulticallHandler()` on the source chain while reusing the same allocator signature. By switching to a different handler, the attacker changes the `_acrossMessageFn`, which results in a different Across deposit message and `relayHash`. This causes the filler's signed claim to become invalid and ultimately enables theft of the filler's first-fill funds.

Impact

Critical

In the first-fill scenario of the Across bridge, the filler can suffer a loss of funds.

Recommendation

It is recommended to include a value in the `mandateHash` that distinguishes between different `_acrossMessageFn` functions.

Remediation

Patched

In the Across arbiter, the 7579 and Multicall paths have been changed to execute via separate, split contracts. Since the arbiter address is used as the `claimant` argument when calling The

Compact's `claim`, and the claim fails if that address changes, a malicious sponsor can no longer perform a claim using a different Across message.

#5 Rhinestone-2511-05 Replayable `targetOps` due to missing `targetChainId` validation in

`AcrossArbiter.across_permit2_erc7579Intent_handleFill()`

ID	Summary	Severity
Rhinestone-2511-05	Because <code>targetAttributesHash</code> is not recomputed and the digest in the Permit2 flow does not include <code>block.chainId</code> , a malicious attacker can replay the same <code>targetOps</code> across multiple chains, allowing repeated execution for any account that has installed <code>Permit2Executor</code> .	High

Description

A malicious attacker can replay the `targetOps` executed in `across_permit2_erc7579Intent_handleFill()` across multiple chains. In `_executePermit2Intent()`, `mandateStub.targetAttributesHash` is assigned directly from `context.targetAttributesHash` instead of being computed with `hashTargetAttributesRaw()`. In `executeTargetOpsWithPermit2Stub()`, the digest is computed as `bytes32 digest = _permit2HashTypedData(permit2Hash, mandateStub.notarizedChainId);` and `block.chainId` is never used. An attacker can execute the same `targetOps` on multiple chains for an account that has installed `Permit2Executor`.

Impact

High

Attackers can replay identical `targetOps` across multiple chains, leading to unauthorized multi-chain execution and potential fund loss for accounts using `Permit2Executor`.

Recommendation

It is recommended to use `block.chainId` instead of `mandateStub.notarizedChainId` when computing the `digest` in `Permit2Executor.executeTargetOpsWithPermit2Stub()`.

Remediation

Patched

The issue has been resolved as recommended.

#6 Rhinestone-2511-06 Incorrect across message function used in `across_permit2_multicall_handleFill()`

ID	Summary	Severity
Rhinestone-2511-06	<code>across_permit2_multicall_handleFill()</code> calls the wrong across-message function, leading to incorrect behavior in the multicall execution flow.	Medium

Description

Although `across_permit2_multicall_handleFill()` is part of the multicall execution flow on the destination chain, it currently uses the `_toAcrossMessageClaimHash` function, which is intended for the ERC7579 intent execution flow. It is recommended to use `_toAcrossMessageMultiCall()` instead of `_toAcrossMessageClaimHash()` in `across_permit2_multicall_handleFill()`.

Impact

Medium

The multicall flow in `across_permit2_multicall_handleFill()` may not function correctly.

Recommendation

It is recommended to use `_toAcrossMessageMultiCall()` instead of `_toAcrossMessageClaimHash()` in `across_permit2_multicall_handleFill()`.

Remediation

Patched

In the Across Arbiter, the two execution paths, ERC-7579 and multicall, were split into separate contracts. The issue was resolved by implementing `_acrossMessageFn()` independently in each separated contract.

#7 Rhinestone-2511-07 Incorrect notarizedChainId usage in handleCompact_ExogenousChain() for EcoArbiter and RelayArbiter

ID	Summary	Severity
Rhinestone-2511-07	PreClaim may fail unexpectedly in EcoArbiter and RelayAdapter due to incorrect arguments passed to handleCompact_ExogenousChain().	Medium

Description

In AcrossArbiter, handleCompact_ExogenousChain() passes order.notarizedChainId as the notarizedChainId argument when calling _compactPreClaimOps(), whereas EcoArbiter and RelayAdapter incorrectly use block.chainId instead of order.notarizedChainId. When block.chainId is used in _compactPreClaimOps(), CompactIntentExecutor.executePreClaimOpsWithCompactStub() computes the digest based on block.chainId rather than notarizedChainId, which may cause signature verification to fail and result in unintended preclaim failure.

Additionally, when calling _compactPreClaimOps(), the elementOffset should be set to elementIndex + 1 instead of 0. If the elementOffset value is incorrect, executePreClaimOpsWithCompactStub() will compute an incorrect allElementsHash, which likewise leads to preclaim signature verification failure.

Impact

Medium

PreClaim may not execute correctly in EcoArbiter and RelayAdapter during handleCompact_ExogenousChain().

Recommendation

In EcoArbiter and RelayAdapter, it is recommended to pass order.notarizedChainId as the notarizedChainId argument and to use elementIndex + 1 as the elementOffset argument when calling handleCompact_ExogenousChain().

Remediation

Patched

The issue has been resolved as recommended.

#8 Rhinestone-2511-08 A malicious filler can tamper with preClaimSig in PreClaimExecution._handlePreClaimOpsCompactERC7579() to deliberately cause the pre-claim call to fail

ID	Summary	Severity
Rhinestone-2511-08	A malicious filler can omit the valid <code>preClaimSig</code> and intentionally trigger preclaim failure, allowing the preclaim step to be bypassed.	High

Description

When calling `PreClaimExecution._handlePreClaimOpsCompactERC7579()`, a malicious filler can remove the valid `signature.preClaimSig` and still invoke the function. In that case, even if the preclaim fails, the transaction continues to execute. As a result, the filler can deliberately cause signature verification to fail and effectively skip the preclaim process.

Impact

High

A malicious filler can intentionally cause the preclaim step to fail.

Recommendation

It is recommended to include `preClaimSig` in the `mandateHash`.

Remediation

Patched

`CompactIntentExecutor.executePreClaimOpsWithCompactStub()` now returns `sigOk` and `execOk`, indicating whether a signature verification failure or an execution failure occurred. In `PreClaimExecution._handlePreClaimOpsCompactERC7579()`, the transaction is now

reverted in the fill-first scenario when `okSig` is `false` and `okGas` is `true`, preventing the filler from skipping PreClaim by injecting an intentionally invalid signature.

#9 Rhinestone-2511-09 Incorrect balance check in MultiCaller._withdrawApprovedTokens

ID	Summary	Severity
Rhinestone-2511-09	MultiCaller.multiCallWithDrainToken() fails for native tokens because _withdrawApprovedTokens() uses an inverted balance comparison, causing withdrawals to revert unless the remaining native balance matches exactly.	Low

Description

When `multiCallWithDrainToken()` is called in `MultiCaller`, all intended multicalls execute first, and any remaining balance is sent to the `recipient` through `_withdrawApprovedTokens()`. However, when the token is `NATIVE_TOKEN`, the function checks: `require(amount >= address(this).balance, WithdrawFailed());`

This comparison is incorrect. It requires `amount == address(this).balance` for the withdrawal to succeed, because any value of `amount` that is not equal to the contract's full native balance causes the condition to fail. As a result, if the leftover native token amount after executing the multicall does not exactly match the contract's full native balance, `multiCallWithDrainToken()` will always revert.

Impact

Low

`multiCallWithDrainToken()` cannot successfully withdraw leftover native tokens unless the remaining amount exactly matches the contract's full balance.

Recommendation

It is recommended to replace the condition with: `require(amount <= address(this).balance, WithdrawFailed());`

Remediation

Patched

The issue has been resolved as recommended.

#10 Rhinestone-2511-10 A malicious filler can bypass the Router and interact directly to selectively execute intents, omit targetOps, or trigger a refund.

ID	Summary	Severity
Rhinestone-2511-10	Because Eco Bridge does not enforce fill through the Router, a malicious filler can directly call <code>EcoPortal.fill()</code> to partially execute intents, omit <code>targetOps</code> , or force refunds, breaking atomicity and allowing token-only fills without performing the required operations.	High

Description

Eco Bridge does not have an exclusive relayer like Across Bridge, so a malicious filler can bypass the Router and call `EcoPortal.fill()` directly to selectively process requests from different chains, such as handling only deposits that occurred on the exogenousChain or only those on the notarizedChain. In this case, because a single `intentHash` has already been filled by the attacker, any legitimate filler attempting to atomically fill all deposits through the Router will always fail. Therefore, the remaining intents that were not filled will be refunded, degrading the user experience.

Additionally, when a claim is executed in `EcoArbiter` and the `targetOps` type is `ERC7579` or `Eip712Hash`, `targetOps` is not included in the intent's `route.calls` (`IntentLib._encodeTargetExecutions`). Therefore, a malicious filler that fills without going through the Router can omit `targetOps`, perform only the token transfer, and still be compensated. In this case, the user has no guarantee that `targetOps` will be executed.

Impact

High

An attacker can selectively fill intents in a way that prevents legitimate atomic fills through the Router, causing remaining intents to be refunded, degrading UX, and breaking expected execution

guarantees. Moreover, the attacker can skip `targetOps` entirely while still receiving compensation, leaving users with incomplete or unexecuted operations.

Recommendation

A validation call must be added to the intent to ensure that the filler cannot bypass the router and call the Eco bridge directly, by verifying that the intent is being executed within the context of Eco Arbiter's fill flow.

Remediation

Patched

In Eco Arbiter, the intent creation flow has been patched so that the call to `exclusivityCallback()` on `exclusivityCallbackTarget` is included in `route.calls`. Since `EcoExclusivity.exclusivityCallback()` enforces that the `withEcoExclusiveFlag()` modifier is executed first, it forces fillers to enter through `eco_handleFill()`.

#11 Rhinestone-2511-11 Collision between `target` address and `singleCall()` function signature in `Caller.sol`

ID	Summary	Severity
Rhinestone-2511-11	<code>SingleCaller</code> is designed to use a fallback-based “single call” mechanism where the first 20 bytes of calldata are interpreted as the target address. However, if the first 4 bytes of that target address collide with the <code>singleCall()</code> function selector (<code>0x9280836c</code>), the call is dispatched to <code>singleCall()</code> instead of the fallback, causing an unintended revert.	Medium

Description

In the current design, the “single call mechanism” operates via `fallback()` , where the first 20 bytes of calldata are interpreted as the target address and the remaining bytes are forwarded to the target.

However, due to the EVM's function dispatch logic, the first 4 bytes of calldata are always used as the function selector. Therefore, even if a user constructs calldata in the form of `[target (20 bytes)][callData (...)]` to trigger the fallback path, if the first 4 bytes of that calldata (= the top 4 bytes of the target address) match the selector of `singleCall()` , `0x9280836c` , the contract will dispatch to `singleCall(address, bytes)` instead of hitting the fallback. Since `singleCall()` always reverts with `UseFallbackInstead()` , normal fallback-based forwarding becomes impossible.

Impact

Medium

If the first 4 bytes of the target address match the `singleCall()` selector (`0x9280836c`), the call is dispatched to `singleCall()` instead of `fallback()` , causing an unintended revert and preventing fallback-based forwarding to that target.

Recommendation

It is recommended to remove `SingleCaller.singleCall()` from `Caller.sol` to eliminate selector collision and ensure the fallback-based forwarding mechanism is always reachable.

Remediation

Patched

The issue has been resolved as recommended.

#12 Rhinestone-2511-12 Reentrancy in PreClaim via routeClaim, allowing Compact claim execution in advance and causing CLAIM failure under fill-first mode

ID	Summary	Severity
Rhinestone-2511-12	In a fill-first flow, a malicious sponsor can cause Compact's <code>multichainClaim()</code> to be executed during PreClaim via reentrancy, so that the later "normal" Claim path fails when <code>_unlockNotarizedChain()</code> is reached again. This makes the Claim revert entirely, leaving the filler out-of-pocket while the user can later be refunded via forced withdrawal, enabling a practical double-spend.	High

Description

`Router.routeClaim()` can be reentered during the PreClaim phase via `_handlePreClaimOpsMulticall()`, allowing the reentrant call to execute the claim-unlock step early. When control returns to the original `routeClaim()` execution, the normal Claim flow later reaches `_unlockNotarizedChain()` again, but the unlock operation has already been performed in the reentrant call. Since the same claim step cannot be executed twice, `_unlockNotarizedChain()` reverts and the entire Claim transaction fails.

1. The attacker (sponsor) submits a fill request and configures PreClaim to call `A.attack()` via `_handlePreClaimOpsMulticall()`.
2. On the destination chain, the filler executes fill-first, and the attacker receives the funds.
3. After confirming fill-first has succeeded, the attacker calls `A.set()` to store `relayerContexts` and `adapterCallDataes`, matching what would normally be passed to `Router.routeClaim()`.
4. The filler calls `Router.routeClaim()`:

- During `PreClaim ( _compactPreClaimOps() )`, `A.attack()` is invoked.
 - `A.attack()` reenters `Router.routeClaim()` and triggers `_unlockNotarizedChain()` early inside the reentrant execution.
 - After reentrancy ends, the original Claim flow later reaches `_unlockNotarizedChain()` again, which now reverts because the action has already been performed.
 - The entire Claim transaction reverts.
5. The attacker calls `enableForcedWithdrawal()` in The Compact. If Claim does not succeed within `resetPeriod`, the attacker calls `forcedWithdrawal()` to recover/refund, completing a double-spend against the filler.

A proof-of-concept sketch (simplified) illustrates the reentrant call pattern:

```
contract A {
    bool used;
    address owner;
    Router router;
    bytes relayerContexts;
    bytes adapterCalldataaes;

    constructor() {
        owner = msg.sender;
    }

    function attack() public {
        if (used) return;
        if (adapterCalldataaes.length == 0) return;
        used = true; // To avoid infinite loops

        bytes;
        bytes;

        _relayerContexts[0] = relayerContexts;
        _adapterCalldataaes[0] = adapterCalldataaes;

        router.routeClaim(_relayerContexts, _adapterCalldataaes);
    }

    function set(bytes calldata _relayerContexts, bytes calldata _adapterC
allldataaes) public {
        require(msg.sender == owner);
    }
}
```

```
        relayContexts = _relayContexts;  
        adapterCallDataes = _adapterCallDataes;  
    }  
}
```

While a filler could try to constrain `gas` and `preClaimGasStipend` to exactly `minGas` so `PreClaim` runs out of gas and avoids the exploit, this is not a reliable defense: it is hard for fillers to consistently calibrate this during `resetPeriod`, and using higher gas limits makes the attack more feasible.

Impact

High

A malicious sponsor can deliberately force the `Claim` to revert after having already received funds from a fill-first execution, and then later use `Compact`'s forced withdrawal path to trigger a refund, resulting in a double-spend where the filler loses funds.

Recommendation

It is recommended to add a `nonReentrant` modifier (or equivalent reentrancy guard) to `RouterLogic.routeClaim()` to prevent reentrant execution during `PreClaim`.

Remediation

Patched

The issue has been resolved as recommended.

#13 Rhinestone-2511-13 `handleNative()` performs no balance validation when `msg.value` is 0

ID	Summary	Severity
Rhinestone-2511-13	<code>handleNative()</code> only performs Router native-balance validation when <code>msg.value > 0</code> . If <code>msg.value == 0</code> , the modifier executes without any balance check, enabling payable adapter paths to move native tokens already held by the Router without detection.	Low

Description

`handleNative()` is intended to ensure that ETH accounting is correct after a call that supplies native tokens. It snapshots `balanceBefore`, executes the function body, and then checks the final Router balance against the expected value. However, the modifier skips the check entirely when `msg.value == 0`:

```
modifier handleNative() {
    uint256 msgValue = msg.value;
    if (msgValue == 0) {
        -;
    } else {
        uint256 balanceBefore = address(this).balance;
        -;
        uint256 expected = balanceBefore - msgValue;
        require(expected == address(this).balance, UnusedETH());
    }
}
```

This creates an unvalidated execution path for calls with `msg.value == 0`. If the Router holds any ETH, an attacker can call `RouterLogic.optimized_routeFill921336808()` with `msg.value = 0` and then invoke payable adapter logic such as

`MultiCallAdapter.multicall_handlePayable()` to drain native tokens from the Router, bypassing `handleNative()`'s intended protection.

Impact

Low

If the Router holds any native tokens, an attacker can drain those funds by using payable adapter execution paths while bypassing `handleNative()` checks when `msg.value == 0`.

Recommendation

It is recommended that `HandleSafeFillETH.handleNative()` performs balance validation even when `msg.value == 0`. Additionally, replace the strict equality check with a safer inequality condition to avoid reverting when the Router balance increases unexpectedly:

- Perform validation for all calls (including `msg.value == 0`)
- Use `require(expected <= address(this).balance)` instead of `require(expected == address(this).balance)`

Remediation

Patched

A grieving/DoS vector exists with `handleNative`'s strict balance check: if `targetOps` calls an external contract and that contract is malicious, it can force the fill to fail by sending even 1 wei back to the Router, causing the post-check to revert. Because the Router is not intended to hold funds and to avoid introducing this grieving surface, the patch removed `handleNative()` entirely.

#14 Rhinestone-2511-14 Minor Suggestions

ID	Summary	Severity
Rhinestone-2511-14	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

- `EcoArbiter` and `RelayArbiter` need to add this validation in `handleCompact_ExogenousChain()`: `require(order.targetOps.hashEIP712() == Constants.NO_EXEC, InvalidTargetOps());`

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

It has been patched as recommended.

Revision History

Version	Date	Description
1.0	Jan 16, 2026	Initial version
1.1	Jan 20, 2026	Added post-patch commits

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

